

## Cloud infrastructure

[Menu](#)[On this page](#)

# Séance 3

## Accès distant

### Objectifs

- Mettre en place l'accès distant à k8s via un token.
- Configurer l'espace personnel.

## Laboratoire

### Tâche 1

Création d'un [NameSpace](#) individuel sur l'environnement *kubernetes* partagé à l'aide de la commande `kubectl create namespace`

| Exigence                                                                |
|-------------------------------------------------------------------------|
| Le <i>namespace</i> doit être nommé <code>ns-&lt;shortname&gt;</code> . |

### Question

Quelle commande `kubectl` faut-il utiliser ?

Listez les différents *NameSpace* à l'aide de la commande `kubectl`.

### Tâche 2

Création d'un [ServiceAccount](#) individuel dans le *namespace* individuel sur l'environnement *kubernetes* partagé à l'aide de la commande `kubectl create serviceaccount` .

### Exigence

Le *ServiceAccount* doit être nommé `sa-<shortname>` .

### Question

Quelle commande kubectl faut-il utiliser ?

Listez les différents *ServiceAccount* dans le *Namespace* individuel ainsi que sur l'ensemble de l'environnement *kubernetes* à l'aide de la commande kubectl.

Création d'un [ClusterRoleBinding](#) pour assigner le *ClusterRole* `cluster-admin` pré-existant au *ServiceAccount* précédemment créé à l'aide de la commande `kubectl create clusterrolebinding` .

### Exigences

Le *ClusterRoleBinding* doit être nommé `cluster-admin-<shortname>` .

Le *ClusterRole* à utiliser est `cluster-admin`

### Question

Quelle commande kubectl faut-il utiliser ?

Création d'un [secret](#) - de manière déclarative - lié au *ServiceAccount* précédemment créé pour obtenir un *token* d'accès distant.

### Exigences

Le *Secret* doit être nommé `<shortname>-token` .

Le *Secret* doit être de type `kubernetes.io/service-account-token`

## Exemple de secret Kubernetes

yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: sa-token
annotations:
```

```
kubernetes.io/service-account.name: sa
type: kubernetes.io/service-account-token
```

### Question

Quel est le contenu du fichier yaml correspondant au secret et quelle commande kubectl faut-il utiliser pour l'appliquer ?

*L'objectif est de créer un accès à l'environnement kubernetes via un token pour permettre son utilisation depuis un ordinateur distant.*

## Tâche 3

Installation de [kubectl](#) sur un ordinateur distant afin d'interagir avec l'environnement *kubernetes* partagé via le *ServiceAccount* crée précédemment.

Pour créer le fichier de configuration nécessaire à *kubectl* sur l'ordinateur distant il est nécessaire de récupérer le fichier kubeconfig du serveur à l'aide de la commande `kubectl config view --raw` et d'y réaliser des modifications telles que ; le nom ou l'IP du serveur, le nom d'utilisateur à utiliser ainsi que le token.

Le token peut être récupéré en affichant le contenu du *secret* créé lors de la tâche précédente à l'aide de la commande `kubectl get secret <secret name> -o yaml` .

### ATTENTION

Attention, dans un *secret* les données sont encodées en base64.

## Exemple de fichier kubeconfig utilisant un token

```
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: {omitted}
    server: https://127.0.0.1:6443
  name: default
contexts:
- context:
    cluster: default
    user: my-user
```

yaml

```
name: default
current-context: default
kind: Config
preferences: {}
users:
- name: my-user
  user:
    token: {omitted}
```

La clé `clusters[0].cluster.certificate-authority-data` doit contenir le *CertificateAuthority* de l'environnement *kubernetes* au format base64.

La clé `users[0].user.token` doit contenir le token de l'utilisateur non encodé au format base64.

## Format YAML

Le format YAML (*YAML Ain't Markup Language*) est largement utilisé pour la configuration dans Kubernetes (et ailleurs).

Sa structure repose sur des clés et des valeurs, organisées de manière hiérarchique grâce à l'**indentation**.

Il est essentiel d'utiliser des **espaces** pour l'indentation (jamais de tabulations), car une mauvaise indentation peut rendre le fichier invalide.

Chaque clé est suivie de `:` et peut contenir une valeur simple ou une structure imbriquée.

Il est possible d'accéder à une valeur imbriquée via un « accès à plat » en utilisant la notation pointée, par exemple : `parent.child` .

Exemple :

```
parent:
  child: valeur
  list:
    - item1
    - item2
```

yaml

`parent.child` vaut `valeur` tandis que `parent.list[0]` vaut `item1` .

- `parent.list` fait référence à la liste entière ( `[item1, item2]` )
- `parent.list[0]` permet d'accéder au premier élément de cette liste, soit `item1` .
- on pourrait également écrire

```
parent:
  child: valeur
  list: ['item1', 'item2']
```

*N'hésitez pas à revoir la partie consacrée à YAML dans le cours 4DOP1dr.*

Il est nécessaire de définir la variable d'environnement `KUBECONFIG` avec le *path* vers votre fichier de configuration.

| Shell      | Commande                                                                          |
|------------|-----------------------------------------------------------------------------------|
| PowerShell | <code>\$env:KUBECONFIG = "C:\Users\myuser\kubeconfig-sa-myuser-token.yaml"</code> |
| Bash       | <code>export KUBECONFIG=/home/myuser/kubeconfig-sa-myuser-token.yaml</code>       |

### Question

Listez les différents *NameSpace* et *pod* à l'aide de la commande `kubectl` .

*L'objectif est d'accéder à l'environnement kubernetes depuis un ordinateur disant.*

Mise à jour le: lundi 17 novembre 2025 à 06:39

Previous page  
[Séance 2](#)

Next page  
[Séance 4](#)